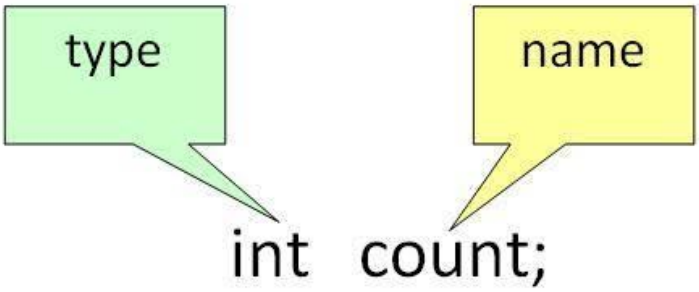
Variables in Java

A variable is a name given to a memory location. It is the basic unit of storage in a program.

- The value stored in a variable can be changed during program execution.
- A variable is only a name given to a memory location, all the operations done on the variable effects that memory location.
- In Java, all the variables must be declared before use

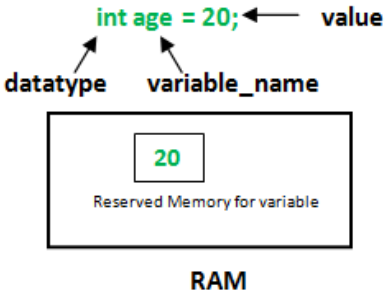
How to declare variables?



type: Type of data that can be stored in this variable.
name: Name given to the variable.
In this way, a name can only be given to a memory location. It can be assigned values in two ways:

- Variable Initialization
- Assigning value by taking input

How to initialize variables?



datatype: Type of data that can be stored in this variable.

variable_name: Name given to the variable.

value: It is the initial value stored in the variable

TYPES OF VARIABLES

- Local Variables
- Instance Variables
- Static Variables

Local Variables: A variable defined within a block or method or constructor is called local variable.

- These variable are created when the block in entered or the function is called and destroyed after exiting from the block or when the call returns from the function.
- The scope of these variables exists only within the block in which the variable is declared. i.e. we can access these variable only within that block.
- Initilisation of Local Variable is Mandatory.

```
public class StudentDetails {

public void StudentAge(){

// local variable age
int age = 0;
age = age + 5;
System.out.println("Student age is : " + age);
}

public static void main(String args[]){

StudentDetails obj = new StudentDetails();
obj.StudentAge();
}
}
```

OUTPUT : **Student age is : 5**

Instance Variables: Instance variables are non-static variables and are declared in a class outside any method, constructor or block.

- As instance variables are declared in a class, these variables are created when an object of the class is created and destroyed when the object is destroyed.
- Unlike local variables, we may use access specifiers for instance variables. If we do not specify any access specifier then the default access specifier will be used.
- Initilization of Instance Variable is not Mandatory. Its default value is 0
- Instance Variable can be accessed only by creating objects.

```
class Marks {
   // These variables are instance variables.
   // These variables are in a class
   // and are not inside any function
   int engMarks;
   int mathsMarks;
   int phyMarks;
}

class MarksDemo {
   public static void main(String args[])  {
      // first object
      Marks obj1 = new Marks();
      obj1.engMarks = 50;
      obj1.mathsMarks = 80;
      obj1.phyMarks = 90;

      // second object
      Marks obj2 = new Marks();
      obj2.engMarks = 80;
      obj2.mathsMarks = 60;
      obj2.phyMarks = 85;

      // displaying marks for first object
      System.out.println("Marks for first object:");
      System.out.println(obj1.engMarks);
      System.out.println(obj1.mathsMarks);
      System.out.println(obj1.phyMarks);

      // displaying marks for second object
      System.out.println("Marks for second object:");
      System.out.println(obj2.engMarks);
```

```
        System.out.println(obj2.mathsMarks);
        System.out.println(obj2.phyMarks);
    }
}
```

Output:

```
Marks for first object:
50
80
90
Marks for second object:
80
60
85
```

Static Variables: Static variables are also known as Class variables.

- Static variables are also known as Class variables which are declared with the “static” keyword in a class.
- A single copy of each variable per class is to be shared by all instances of the class.
- Static variables are stored in static memory. Static variables are rarely used other than it is declared final and used as either public or private constants.
- When the program starts static variables are created and when the program stops, static variables are destroyed.
- Default values of static variables are same as instance variables. For numbers the default value is 0, for Boolean the default value is false and for object references the default value is null.
- When static or class variables are declared as public static final, then variables names are all in upper case. The naming syntax is the same as instance and local variables in case of static variables are not declared public and final.
- Static variables can be accessed by calling with the class name. It does not need any object. Eg: Classname.staticVariableName

- Static variables are initialized only one time, i.e. at the start of the execution. These variables will be initialized first, before the initialization of any instance variables.

```
class Student{
   int rollno;
   String name;
   static String college ="CMS"; //Static Variable gets memory once
   /*Constructor of Student class*/
   Student(int r,String n) {
      rollno = r;
      name = n;
   }
   /*Method For Displaying Student Details*/
   void display(){
// print the value of roll no, name and college
      System.out.println(rollno+" "+name+" "+college);
 }
   public static void main(String args[]) {
      Student s1 = new Student(101,"ANTO");
      Student s2 = new Student(102,"ANOOP");
      s1.display();  // call the display function using the s1 object
      s2.display();   // call the display function using the s2 object
   }
}
```

Output:

```
101 ANTO CMS
102 ANOOP CMS
```